DRISHTI
A Revolutionary Concept

MakerNova 4.0

Detailed Report On

AI/ML-BASED BIODEGRADABLE AND NON-BIODEGRADABLE MATERIAL CLASSIFICATION

TEAM MEMBERS	MENTORS
Name	**Mentor Name**
Aaditya	Awadhi Jain
Aashika Venkatesan Sudha	Heet Lunagariya
Heshika Pareek	Rohansh Dattani
Kamlesh Prajapati	
Keya Mehta	
Krishna Tekwani	
Lucky Makwana	
Neelam Jat	
Shubham Khunt	
Sudarshan Gaggar	
Tushar Patil	

Acknowledgement

We express our heartfelt gratitude to our academic mentors for their constant guidance, constructive feedback, and invaluable support throughout the course of this project. Their insights have been instrumental in deepening our understanding of artificial intelligence, machine learning, and their applications in sustainable waste management.
We are equally thankful to our institute and all those who provided us with the necessary resources, technical support, and encouragement to design, develop, and test our biodegradable and non-biodegradable material classification model. Their support has been invaluable in the successful completion of this work.
Finally, we extend our appreciation to the researchers and authors whose work in the fields of waste management, computer vision, and deep learning has provided a strong foundation for our project. Their contributions have inspired and guided our efforts toward developing an intelligent system for efficient and automated material classification.

Abstract

The project develops an image-based binary classification system that identifies waste as BIODEGRADABLE or NON-BIODEGRADABLE. Exploratory Data Analysis (EDA) was performed before model development to verify dataset structure, class distribution, image quality, and image properties. The complete workflow covers Python-based dataset handling and EDA, a custom convolutional neural network implemented in PyTorch, transfer learning and fine-tuning using EfficientNet-B0, classification experiments using YOLOv5n, YOLOv8n, YOLOv11n, and YOLOv26n, classical HSV-based color detection using OpenCV, Raspberry Pi deployment, and a Streamlit web application.

Contents

1 Introduction 8

2 Objective and Dataset Overview 8

3 Exploratory Data Analysis and Software Workflow 8
- 3.1 Python Programming . . . 8
- 3.2 NumPy . . . 8
- 3.3 Pandas . . . 8
- 3.4 Matplotlib and Seaborn . . . 8
- 3.5 Exploratory Data Analysis . . . 8
- 3.6 Dataset and Image Statistics . . . 9
- 3.7 EDA Conclusion . . . 9

4 Core Concepts: ML, Neural Networks and the Perceptron 9
- 4.1 Machine Learning . . . 9
- 4.2 Neural Networks . . . 9
- 4.3 The Perceptron . . . 10

5 Custom CNN Classification (PyTorch) 10
- 5.1 Model Architecture (MyNN) . . . 10
- 5.2 Dataset and Preprocessing . . . 10
- 5.3 Training Configuration . . . 11
- 5.4 Training Summary . . . 11
- 5.5 Performance Metrics . . . 11
- 5.6 Confusion Matrix . . . 11
- 5.7 Real-World Inference Testing . . . 11

6 Transfer Learning vs. Fine-Tuning 12
- 6.1 Transfer Learning . . . 12
- 6.2 Fine-Tuning . . . 12
- 6.3 Comparison and Conclusion . . . 13

7 YOLOv5n Classification 13
- 7.1 Model and Training Configuration . . . 13
- 7.2 Performance Metrics . . . 13
- 7.3 Real-World Inference Testing . . . 14

8 YOLOv8 Classification 15
- 8.1 Model and Training Configuration . . . 15

9 YOLOv11 Classification 16

10 YOLOv26 Classification 17

11 YOLO Model Comparison 18
- 11.1 Why YOLOv8 Over the Others . . . 18

12 OpenCV RGB Color Detection 18
- 12.1 Processing Pipeline . . . 18
- 12.2 Results . . . 18
- 12.3 Observations and Limitations . . . 18

13 Raspberry Pi Deployment for Model Inference 19
- 13.1 Introduction to Raspberry Pi . . . 19
- 13.2 Basic Setup . . . 19
- 13.3 SSH (Secure Shell) Overview . . . 19
- 13.4 Model Transfer . . . 19

13.5 Model Deployment and Inference Process 19
13.6 Deployment Flow 19
13.7 Deployment Conclusion 19

14 Biodegradable vs. Non-Biodegradable Classifier: Streamlit Web Application 20
14.1 Components 20
14.2 What is Streamlit and Why It Is Used 20
14.3 Key Functions and Actual Values Used 20
14.4 Test Results: Plastic Bag Image 20
14.5 Test Results: Chili Plant Image 20

15 Conclusion 21

List of Figures

1 Overall distribution of the two waste classes. . . . 9
2 Training/testing composition for each waste class. . . . 9
3 Key image statistics calculated during EDA 9
4 Custom CNN test confusion matrix. . . . 11
5 CNN Test-image 1 12
6 CNN Test-image 2 12
7 Transfer-learning test confusion matrix. . . . 12
8 Fine-tuning test confusion matrix. . . . 13
9 YOLOv5n confusion matrix image. . . . 14
10 YOLOv5n real-world inference example 1. . . . 14
11 YOLOv5n real-world inference example 2. . . . 14
12 YOLOv8 confusion matrix image included in the report. . . . 15
13 YOLOv8 inference image 1. . . . 15
14 YOLOv8 inference image 2. . . . 15
15 YOLOv11 training-run confusion matrix. . . . 16
16 YOLOv11 inference image 1. . . . 16
17 YOLOv11 inference image 2. . . . 16
18 YOLOv26 confusion matrix. . . . 17
19 YOLOv26 inference image 1. . . . 17
20 YOLOv26 inference image 2. . . . 17
21 Annotated image produced by the OpenCV color-detection pipeline. . . . 18
23 Raspberry Pi inference image 1. . . . 19
24 Raspberry Pi inference image 2. . . . 20
25 Raspberry Pi inference image 3. . . . 20
26 Streamlit Test 1 uploaded plastic bag image. . . . 20
27 Streamlit Test 1 prediction output. . . . 20
28 Streamlit Test 2 uploaded chili plant image. . . . 20
29 Streamlit Test 2 prediction output. . . . 20

List of Tables

1 Dataset overview. . . . 8
2 EDA checks and actual outputs. . . . 9
3 Overall image statistics . . . 9
4 Summary of core learning concepts. . . . 10
5 Custom MyNN architecture . . . 10
6 Custom CNN dataset and preprocessing . . . 10
7 Custom CNN training configuration . . . 11
8 Selected epochs from custom CNN training . . . 11
9 Custom CNN accuracy by split . . . 11
10 Custom CNN test-set classification report (409 images) . . . 11
11 Custom CNN real-world inference results . . . 11
12 EfficientNet-B0 transfer-learning configuration . . . 12
13 Transfer-learning performance metrics . . . 12
14 Transfer-learning classification report . . . 12
15 EfficientNet-B0 fine-tuning configuration . . . 12
16 Fine-tuning performance metrics . . . 13
17 Fine-tuning classification report . . . 13
18 YOLOv5n configuration . . . 13
19 Official YOLOv5n evaluation . . . 13
20 Manual YOLOv5n evaluation using sklearn . . . 13
21 Full manual YOLOv5n classification report . . . 13
22 YOLOv5n manual confusion matrix counts . . . 14
23 YOLOv5n model characteristics . . . 14
24 YOLOv5n real-world inference . . . 14
25 YOLOv8n configuration . . . 15
26 YOLOv8 performance metrics . . . 15
27 YOLOv8 per-class test results . . . 15
28 YOLOv11n configuration . . . 16
29 YOLOv11 performance metrics . . . 16
30 YOLOv11 per-class test metrics . . . 16
31 YOLOv26n configuration . . . 17
32 YOLOv26 performance metrics . . . 17
33 YOLOv26 per-class test results . . . 17
34 YOLOv26 test confusion matrix counts . . . 17
35 YOLO model comparison . . . 18
36 OpenCV color-detection setup . . . 18
37 HSV threshold ranges used for color detection . . . 18
38 Useful Raspberry Pi commands . . . 19
39 Streamlit application components . . . 20
40 Streamlit functions and actual values . . . 20

Problem Statement

The increasing accumulation of waste materials and the improper disposal of non-biodegradable products have created significant environmental challenges. Manual segregation of biodegradable and non-biodegradable waste is time-consuming, inefficient, and prone to human error. Therefore, there is a need for an automated and intelligent system capable of accurately identifying different types of materials. This project aims to develop an AI/ML-based image classification system that automatically classifies material images into biodegradable and non-biodegradable categories using computer vision and machine learning/deep learning techniques. The system analyzes visual characteristics such as color, texture, shape, and other image features to distinguish between the two categories. A labeled dataset of material images is used to train and evaluate the classification model. The proposed system can help automate the initial process of waste segregation, reduce manual effort, improve classification efficiency, and support environmentally sustainable waste-management practices.

BIODEGRADABLE & NON-BIODEGRADABLE WASTE IMAGE CLASSIFICATION

1 Introduction

Waste management is an important environmental challenge, and proper segregation of biodegradable and non-biodegradable waste is essential for effective recycling and disposal. Traditional manual sorting is time-consuming and may result in errors.

Artificial Intelligence (AI) and Machine Learning (ML) provide efficient solutions for automatic waste classification using images. In this project, AI/ML techniques are used to analyze images and classify materials into biodegradable and non-biodegradable categories based on their visual features. The system aims to reduce manual effort, improve classification accuracy, and support faster and more efficient waste segregation. The use of image-based classification can help automate the sorting process and reduce human involvement. The proposed approach can also be useful in smart waste-management systems and recycling facilities. Overall, the project demonstrates how AI and ML can be applied to address environmental problems and promote sustainable waste management.

***Index Terms*—Machine Learning, Deep Learning, Image Classification, EDA, CNN, EfficientNet-B0, YOLO, OpenCV, Raspberry Pi, Streamlit.**

2 Objective and Dataset Overview

The project develops an image-based binary classification system that identifies waste as BIODEGRADABLE or NON-BIODEGRADABLE. Exploratory Data Analysis (EDA) was performed before model development to verify dataset structure, class distribution, image quality and image properties.

Dataset Item	Actual Project Value
Total images	25,077
Biodegradable	13,966
Non-biodegradable	11,111
Training images	22,564
Testing images	2,513
Corrupted images	0

TABLE 1
Dataset overview.

3 Exploratory Data Analysis and Software Workflow

3.1 Python Programming

Python was used as the main programming language because the complete workflow - dataset handling, EDA, visualization and model experimentation was implemented in a Google Colab/Python environment. `os` and `os.path.join()` were used to construct and access dataset paths; for example, `TRAIN_PATH` and `TEST_PATH` were created from `DATASET_PATH`. `os.listdir()` was used to inspect folders and verify the two classes, O and R. `os.walk()` was used to inspect the extracted dataset directory structure. `os.path.getsize()` was used to calculate individual image file sizes for EDA.

3.2 NumPy

NumPy was used for numerical operations and array-based processing during image-data analysis. `np.nan` was used when an image property could not be obtained; no such invalid values remained in the final dataset. Array-based calculations supported statistics such as aspect ratio, where aspect_ratio = width/height. Observed aspect-ratio values were mean = 1.309, minimum = 0.133159, and maximum = 8.695652.

3.3 Pandas

Pandas was used to organize image metadata into a DataFrame and calculate dataset statistics. `pd.DataFrame(records)` created the metadata table containing split, original class, class, filename and filepath. The resulting DataFrame shape was (25,077, 5). `df.isnull().sum()` returned 0 missing values in every column. `df['class'].value_counts()` gave BIODEGRADABLE = 13,966 and NON-BIODEGRADABLE = 11,111. `df.groupby(...).agg(['mean','min','max','std'])` produced class-wise statistics for width, height, aspect ratio and file size.

3.4 Matplotlib and Seaborn

Matplotlib and Seaborn were used to convert EDA results into graphs for easier interpretation. `plt.figure(figsize=...)` controlled graph dimensions; `plt.bar()`, `plt.pie()`, `sns.countplot()`, `sns.histplot()` and `sns.boxplot()` were used for visual analysis. `plt.title()`, `plt.xlabel()`, `plt.ylabel()` and `plt.show()` were used for readable presentation. EDA plots were also saved at 300 DPI in `/content/eda_results` for later reporting.

3.5 Exploratory Data Analysis

EDA was performed to check whether the dataset was correctly structured, sufficiently clean and appropriate for image classification. The analysis focused on class balance, split composition, image dimensions, aspect ratio, file size, image mode and corrupted files.

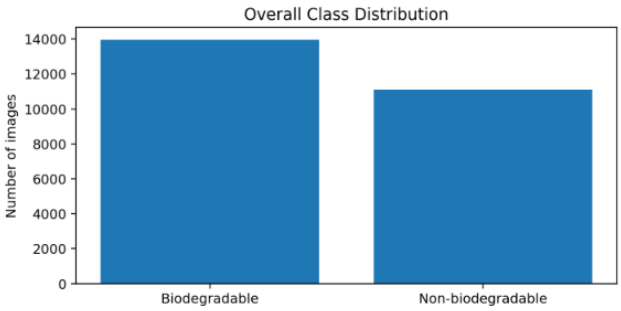


Fig. 1. Overall distribution of the two waste classes.

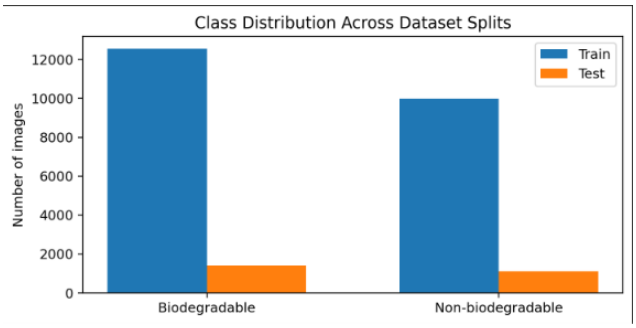


Fig. 2. Training/testing composition for each waste class.

EDA Check	Function / Method	Actual Output
Dataset shape	`df.shape`	(25,077, 5)
Missing values	`df.isnull().sum()`	0 in all 5 columns
Duplicate paths	`duplicated().sum()`	0
Corrupted images	`Image.verify()`	0
Image modes	`value_counts()`	RGB: 24,705; P: 372
Imbalance ratio	class-count ratio	1.257

TABLE 2
EDA CHECKS AND ACTUAL OUTPUTS.

3.6 Dataset and Image Statistics

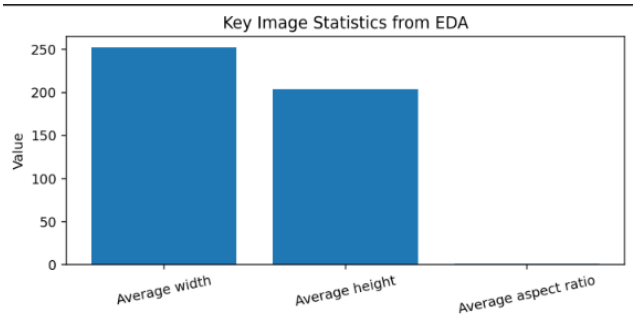


Fig. 3. Key image statistics calculated during EDA

Property	Overall Result
Average width	252.42 pixels
Average height	203.78 pixels
Average aspect ratio	1.309
Average file size	8.65 KB
Width range	51–1,460 pixels
Height range	69–1,500 pixels

TABLE 3
OVERALL IMAGE STATISTICS

The dataset contains 25,077 images across two classes. BIODEGRADABLE contains 13,966 images and NON_BIODEGRADABLE contains 11,111 images. The dataset is moderately imbalanced, with an imbalance ratio of 1.257. This was important to consider while evaluating classification performance. There are no missing metadata values, no duplicate file paths and no corrupted images in the checked dataset. Most images are RGB (24,705), while 372 images are in P mode. This confirms that image-mode handling is relevant during preprocessing. Image dimensions vary considerably, so resizing to a common model input size is required before training.

3.7 EDA Conclusion

The EDA confirmed that the waste image dataset is sufficiently structured for binary image classification. The dataset contains 25,077 images with two target classes, no missing values, no duplicate paths and no corrupted images detected. The analysis of class counts, image dimensions, aspect ratio, file size and image modes provided concrete information for preprocessing and model development.

4 Core Concepts: ML, Neural Networks and the Perceptron

This note explains three foundational concepts behind the CNN model documented separately, and ties each one to how and why it was actually used in that project, with the real results it produced.

4.1 Machine Learning

Machine Learning is the approach of having a program improve its performance on a task by learning patterns from data, rather than being explicitly programmed with fixed rules. This project is a supervised learning task. the model is shown images already labeled biodegradable or non-biodegradable, and it learns a mapping from pixels to class by repeatedly comparing its predictions against these true labels and adjusting itself to reduce the error (loss). Hand-coding rules to distinguish biodegradable vs. non-biodegradable waste from raw pixels (color, shape, texture, packaging) is impractical , ML lets the model discover the relevant visual patterns directly from thousands of labeled examples instead. Learning was measured via CrossEntropyLoss, which fell from 0.5306 (epoch 1) to 0.1765 (epoch 54) on the training set as the model's predictions increasingly matched the true labels, ultimately reaching 92.18% test accuracy.

4.2 Neural Networks

A neural network is a stack of layers of interconnected computational units ("neurons"), where each layer transforms its input into a new representation; layers are trained together via backpropagation, which uses the error at the output to adjust every layer's weights. The custom MyNN architecture is a deep NN , specifically a Convolutional Neural Network (CNN), a variant specialized for images where early layers use convolution to detect local visual patterns (edges, textures) and later layers combine these into higher-level concepts before a final classification

stage. A plain (fully-connected-only) NN would need an impractically large number of parameters to process a 224×224 image directly and would not exploit the spatial structure of images; convolutional layers share weights across the image, letting the network learn location-independent features with far fewer parameters. The network (5 convolutional blocks, 1.72M total parameters) was trained end-to-end via backpropagation using the AdamW optimizer for 54 epochs (early-stopped), reaching 93.10% train accuracy and 92.77% validation accuracy at its best checkpoint (epoch 47), and generalizing to 92.18% accuracy on the unseen test set.

4.3 The Perceptron

The perceptron is the original, simplest neural unit: it computes a weighted sum of its inputs plus a bias, then passes the result through an activation function to produce an output,historically a hard step function for binary decisions, and the conceptual ancestor of every neuron in modern deep networks. Every neuron in MyNN in the convolutional filters and in the fully-connected classifier layers is a generalized perceptron: it performs the same weighted-sum-plus-bias computation, but uses a smooth LeakyReLU activation (instead of a hard step) so gradients can flow during backpropagation. The final Linear(256→2) layer is literally a layer of two perceptron-like units, one scoring "Biodegradable" and one scoring "Non-Biodegradable." A single perceptron can only separate linearly-separable data; stacking many perceptron-like units across multiple layers with non-linear activations in between allows the network to learn the complex, non-linear decision boundary needed to separate real waste images into two classes. The two output perceptron units' raw scores (logits) are converted to probabilities via softmax to produce the model's reported confidence, seen directly in the real-world inference tests, where the two output units produced a 71.84% confidence for Non-Biodegradable on one photo and a 99.72% confidence for Biodegradable on another.

Concept	Where it shows up in the project	Key resulting number
Machine Learning	Supervised training on labeled bio/non-bio images	Test accuracy: 92.18%
Neural Network	The 5-block CNN architecture trained via backpropagation	Best validation accuracy: 92.77% (epoch 47)
Perceptron	Every conv/linear neuron; the final 2-unit output layer	Softmax confidence outputs: 71.84% and 99.72% on real test images

TABLE 4
Summary of core learning concepts.

5 Custom CNN Classification (PyTorch)

This is a custom-built Convolutional Neural Network (CNN) , not a pretrained YOLO/ImageNet backbone trained from scratch in PyTorch to classify images as Biodegradable or Non-Biodegradable. A CNN learns spatial features (edges, textures, shapes) through stacked convolution + pooling layers, progressively building from low-level patterns to high-level object concepts, which makes it well suited to image classification.

5.1 Model Architecture (MyNN)

Block	Layers	Output channels
Conv Block 1	Conv2d(3×3) → BatchNorm2d → LeakyReLU(0.01) → MaxPool2d(2)	64
Conv Block 2	Conv2d(3×3) → BatchNorm2d → LeakyReLU(0.01) → MaxPool2d(2)	64
Conv Block 3	Conv2d(3×3) → BatchNorm2d → LeakyReLU(0.01) → MaxPool2d(2)	128
Conv Block 4	Conv2d(3×3) → BatchNorm2d → LeakyReLU(0.01) → MaxPool2d(2)	256
Conv Block 5	Conv2d(3×3) → BatchNorm2d → LeakyReLU(0.01) → MaxPool2d(2)	512
Pooling	AdaptiveAvgPool2d(1×1)	512
Classifier	Flatten → Linear(512→256) → LeakyReLU → Dropout(0.3) → Linear(256→2)	2 (logits)

TABLE 5
Custom MyNN architecture

Total parameters: 1,721,794 (~1.72M) computed directly from the layer definitions above. BatchNorm2d normalizes activations per batch, stabilizing and speeding up training. LeakyReLU, versus standard ReLU, allows a small negative slope (0.01) so neurons do not fully "die" for negative inputs. AdaptiveAvgPool2d(1,1) collapses each feature map to a single average value, making the network robust to input size and reducing parameters before the classifier. Dropout(0.3) randomly zeroes 30% of activations during training to reduce overfitting.

5.2 Dataset and Preprocessing

Parameter	Value
Dataset	Kaggle:vivekagond/degradable-and-non-degradable-items
Classes	biodegradable (0), non_biodegradable (1)
Split ratios (by code)	Train 70% / Val 20% / Test 10% (via random_split)
Test set size	409 images (Biodegradable: 218; Non-Biodegradable: 191)
Image size	224 × 224
Train augmentation	Random horizontal flip, random rotation (±15°), RandomResizedCrop (scale 0.8–1.0), ColorJitter (brightness/contrast ±0.2)
Val/Test transform	Resize only (no augmentation)
Normalization	ImageNet mean/std: mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]
Batch size	32
GPU	Tesla T4 (×2 available; single GPU used)

TABLE 6
Custom CNN dataset and preprocessing

Data augmentation (flips, rotation, cropping, color jitter) artificially expands the effective training set and reduces

overfitting by exposing the model to varied versions of the same images. ImageNet mean/std normalization is a standard convention that centers pixel values around 0, matching the statistics most pretrained/backbone architectures expect.

5.3 Training Configuration

Parameter	Value
Loss function	CrossEntropyLoss
Optimizer	AdamW, lr=3e-4, weight_decay=1e-4
LR scheduler	ReduceLROnPlateau (mode='min', factor=0.5, patience=2)
Max epochs	75
Early stopping patience	7 (on validation loss)
Epochs actually run	54 (early stopping triggered)

TABLE 7
Custom CNN training configuration

ReduceLROnPlateau halves the learning rate whenever validation loss stops improving for 2 epochs, helping the model fine-tune once progress stalls. Early stopping halts training once validation loss has not improved for 7 consecutive epochs, preventing wasted computation and overfitting , here it triggered at epoch 54.

5.4 Training Summary

Epoch	Train Loss	Train Acc.	Val Loss	Val Acc.	Note
1	0.5306	74.05%	0.4556	80.39%	First epoch
9	0.3800	82.70%	0.3519	85.42%	
26	0.2615	89.11%	0.2484	88.48%	
47	0.1803	93.10%	0.1826	92.77%	Best model saved (lowest val loss)
54	0.1765	93.14%	0.1926	92.40%	Early stopping triggered (7 epochs no improvement)

TABLE 8
Selected epochs from custom CNN training

Training loss decreased steadily and consistently from 0.531 to 0.177 across the run, with train accuracy rising from 74.1% to 93.1%. Validation loss fluctuated epoch-to-epoch (a normal effect of the 20%-sized val set and augmentation randomness) but trended downward overall, bottoming out at epoch 47 (val loss 0.1826, val acc 92.77%). No signs of severe overfitting were observed: at the best epoch, train accuracy (93.10%) and validation accuracy (92.77%) remain close together. The checkpoint saved at epoch 47 (`biodegradable_nonbiodegradable_cnn.pt`) is the one used for all downstream evaluation and real-world testing below.

5.5 Performance Metrics

Split	Accuracy
Train	94.26%
Validation	92.77%
Test	92.18%

TABLE 9
Custom CNN accuracy by split

Class	Precision	Recall	F1-score	Support
Biodegradable	0.9471	0.9037	0.9249	218
Non-Biodegradable	0.8955	0.9424	0.9184	191
Macro avg	0.9213	0.9230	0.9216	409
Weighted avg	0.9230	0.9218	0.9218	409

TABLE 10
Custom CNN test-set classification report (409 images)

Macro average treats both classes equally regardless of size; weighted average weights each class's contribution by its support (image count). since the two classes here are fairly balanced (218 vs. 191), the two averages are close.

5.6 Confusion Matrix

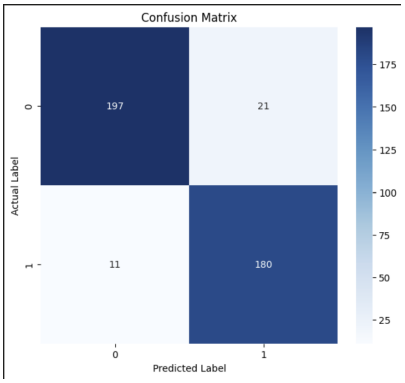


Fig. 4. Custom CNN test confusion matrix.

The test set contains 409 images; label 0 = Biodegradable and label 1 = Non-Biodegradable. A total of 197/218 Biodegradable images were correctly classified (21 misclassified as Non-Biodegradable), and 180/191 Non-Biodegradable images were correctly classified (11 misclassified as Biodegradable). Both error counts are relatively small and roughly balanced between classes, consistent with the precision/recall values above , Biodegradable has slightly higher precision but lower recall, and vice versa for Non-Biodegradable.

5.7 Real-World Inference Testing

we run inference on two real photos taken outside the original dataset, using the same 224×224 resize + ImageNet normalization as the validation transform (no augmentation).

Test image	Predicted class	Confidence
`Fig.6.` (camera photo)	Non-Biodegradable	71.84%
`Fig.5.`	Biodegradable	99.72%

TABLE 11
Custom CNN real-world inference results



Fig. 5. CNN Test-image 1



Fig. 6. CNN Test-image 2

The reported confidence is the softmax probability of the predicted class. the model's own estimate of certainty, not a guarantee of correctness. A confidence of 71.84% versus 99.72% on the second image suggests the model was noticeably less certain about the first, real-world camera photo than about the second, which may reflect a domain gap between the training dataset's images and an actual phone camera photo (different lighting, background clutter, resolution).

A custom 5-block CNN (1.72M parameters) trained from scratch reached 92.18% test accuracy on held-out data, with balanced per-class F1-scores (0.925 Biodegradable, 0.918 Non-Biodegradable).

6 Transfer Learning vs. Fine-Tuning

The backbone used in both was EfficientNet-B0 (ImageNet-pretrained, IMAGENET1K_V1 weights), with an identical custom classifier head: Dropout(0.3) → Linear(→256) → LeakyReLU → Dropout(0.5) → Linear(256→2).

6.1 Transfer Learning

Parameter	Value
Model variant	EfficientNet-B0 (pretrained), entire backbone frozen
Task	Binary image classification
Dataset	Local Biodegradable / Non-Biodegradable image folders (ImageFolder, pre-split)
Classes	Biodegradable, Non-Biodegradable
Test images	405 (204 Biodegradable; 201 Non-Biodegradable)
Image size	224 × 224
Batch size	64
Epochs (configured / actual)	20 / 13 (early-stopped, patience = 5)
Optimizer	Adam, weight_decay = 1e-4, lr = 0.01
Loss function	CrossEntropyLoss
Hardware	NVIDIA GeForce RTX 5050 Laptop GPU (CUDA)

TABLE 12
EfficientNet-B0 transfer-learning configuration

Split	Accuracy
Test	98.27%
Validation	97.78%
Training	97.77%

TABLE 13
Transfer-learning performance metrics

Class	Precision	Recall	F1-score	Support
Biodegradable	0.9950	0.9706	0.9826	204
Non-Biodegradable	0.9709	0.9950	0.9828	201
Weighted avg	0.9830	0.9827	0.9827	405

TABLE 14
Transfer-learning classification report

Best validation loss was 0.0532 (epoch 8); final logged loss at epoch 13 was train 0.1661 and val 0.0641.

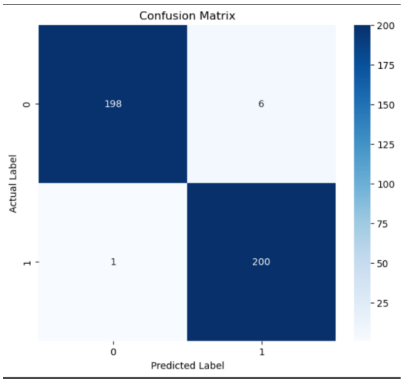


Fig. 7. Transfer-learning test confusion matrix.

On the test set of 405 images, 198/204 Biodegradable images were correctly predicted (6 misclassified), and 200/201 Non-Biodegradable images were correctly predicted (1 misclassified). Only the classifier head was trainable; all EfficientNet-B0 backbone weights stayed frozen throughout. Training ran 13 of 20 configured epochs before early stopping (5 epochs without validation-loss improvement). Single-image inference demos produced 91.81% confidence (Non-Biodegradable) and 95.60% confidence (Biodegradable).

6.2 Fine-Tuning

Parameter	Value
Model variant	EfficientNet-B0 (pretrained), last 2 feature blocks unfrozen (`model.features[-2:]`)
Task	Binary image classification
Dataset	Same dataset/folders as Transfer Learning
Classes	Biodegradable, Non-Biodegradable
Test images	405 (204 Biodegradable; 201 Non-Biodegradable)
Image size	224 × 224
Batch size	64
Epochs (configured / actual)	20 / 8 (early-stopped, patience = 5)
Optimizer	Adam, weight_decay = 1e-4, lr = 0.01
Loss function	CrossEntropyLoss

TABLE 15
EfficientNet-B0 fine-tuning configuration

Split	Accuracy
Test	99.26%
Validation	97.78%
Training	98.21%

TABLE 16
FINE-TUNING PERFORMANCE METRICS

Class	Precision	Recall	F1-score	Support
Biodegradable	0.9902	0.9951	0.9927	204
Non-Biodegradable	0.9950	0.9900	0.9925	201
Weighted avg	0.9926	0.9926	0.9926	405

TABLE 17
FINE-TUNING CLASSIFICATION REPORT

Best validation loss was 0.0516 (epoch 3); final logged loss at epoch 8 was train 0.1054 and val 0.1346.

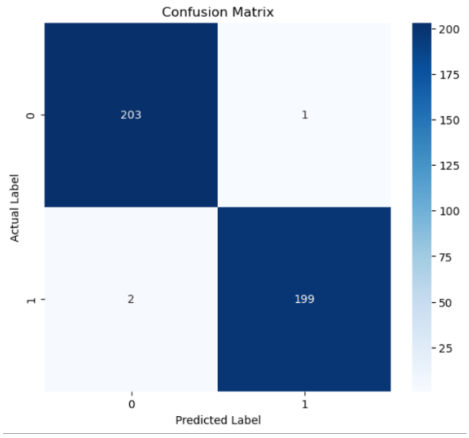


Fig. 8. Fine-tuning test confusion matrix.

On the test set of 405 images, 203/204 Biodegradable images were correctly predicted (1 misclassified), and 199/201 Non-Biodegradable images were correctly predicted (2 misclassified). The classifier head plus the last two backbone feature blocks were trainable; earlier blocks stayed frozen. Training ran only 8 of 20 configured epochs . best validation loss was reached at epoch 3, followed by early stopping. Single-image inference demos produced 83.22% confidence (Non-Biodegradable) and 87.53% confidence (Biodegradable).

6.3 Comparison and Conclusion

Fine-Tuning achieved higher test accuracy than Transfer Learning (99.26% vs. 98.27%), while validation accuracy was identical at 97.78% for both. Fine-Tuning led on every reported weighted-average precision/recall/F1 metric (0.9926 vs. 0.9827–0.9830) and showed more balanced per-class recall (0.9951/0.9900 vs. 0.9706/0.9950). Fine-Tuning reached its best validation loss sooner (epoch 3 vs. epoch 8) and stopped earlier overall (8 vs. 13 epochs). Fine-Tuning updates more parameters (2 backbone blocks + head vs. head only), implying higher per-epoch compute; this was not numerically confirmed. Based strictly on the logged results, Fine-Tuning outperformed Transfer Learning on every reported metric for this dataset, at the likely cost of higher per-epoch computation.

7 YOLOv5n Classification

7.1 Model and Training Configuration

Parameter	Value
Model variant	YOLOv5n-cls (`yolov5n-cls.pt`, pretrained)
Task	Binary image classification
Dataset	Kaggle "Waste Classification Data" (techsash), remapped to 2 classes
Classes	biodegradable, non_biodegradable
Train/Val split	90% / 10% of the original TRAIN folder (train_test_split, random_state=42)
Test images	2,513 (biodegradable: 1,401; non_biodegradable: 1,112)
Image size	224 × 224
Batch size	32
Epochs	15 (completed; no early-stop configured)
Optimizer	Adam, lr0=0.001, weight decay=5e-05, label smoothing=0.1
Augmentation	RandomResizedCrop, HorizontalFlip, ColorJitter, ImageNet Normalize (via Albumentations)
GPU	Tesla T4 (14,913 MiB)
Pretrained weights	`yolov5n-cls.pt`
Training time	0.252 hours (~15 minutes) for 15 epochs the fastest of all models trained on this dataset so far

TABLE 18
YOLOV5N CONFIGURATION

7.2 Performance Metrics

Class	Images	Top-1 Accuracy	Top-5 Accuracy
All	2,513	93.8%	100.0%
biodegradable	1,401	97.1%	100.0%
non_biodegradable	1,112	89.7%	100.0%

TABLE 19
OFFICIAL YOLOV5N EVALUATION

Inference speed was 0.1 ms pre-process, 0.7 ms inference, and 0.0 ms post-process per image.

Metric	Value
Accuracy	90.69%
Precision	86.89%
Recall	92.99%
F1-score	89.83%

TABLE 20
MANUAL YOLOV5N EVALUATION USING SKLEARN

Class	Precision	Recall	F1-score	Support
biodegradable	0.9410	0.8887	0.9141	1,401
non_biodegradable	0.8689	0.9299	0.8983	1,112
Macro avg	0.9050	0.9093	0.9062	2,513
Weighted avg	0.9091	0.9069	0.9071	2,513

TABLE 21
FULL MANUAL YOLOV5N CLASSIFICATION REPORT

The manual evaluation (90.69% accuracy) is noticeably lower than the official classify/val.py result (93.8%). Inspecting the code shows why: the manual script's image transform is only Resize + ToTensor(), without the ImageNet mean/std normalization that was actually used

during training (visible in the training log's Albumentations pipeline); this preprocessing mismatch likely explains the gap.

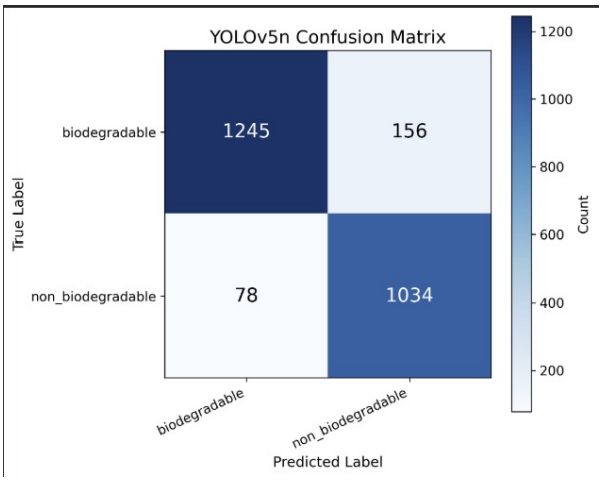


Fig. 9. YOLOv5n confusion matrix image.

Actual / Predicted	biodegradable	non_biodegradable
biodegradable	1,245	156
non_biodegradable	78	1,034

TABLE 22
YOLOv5n manual confusion matrix counts

The confusion matrix shows that 1,245/1,401 biodegradable images were correctly classified (156 misclassified as non_biodegradable), while 1,034/1,112 non_biodegradable images were correctly classified (78 misclassified as biodegradable). The dominant error mode here is biodegradable images being misread as non_biodegradable , notably the opposite pattern from the official val.py breakdown, which showed non_biodegradable as the weaker class (89.7% vs. 97.1%), reinforcing that the missing normalization step in the manual script changes error behavior, not just overall accuracy.

Training loss decreased steadily across all 15 epochs (0.399 → 0.302), but validation ("test") loss and accuracy were noisy: it dipped as low as 0.887 accuracy at epoch 3 before climbing. The best checkpoint (`best.pt`) corresponds to epoch 8 (train loss 0.340, val loss 0.309, top1_acc 93.8%) and matches the official post-training val.py result. After epoch 8, training loss kept falling (0.340 → 0.302) but validation accuracy plateaued/oscillated (93.8% → 92.6% by epoch 15) without surpassing the epoch-8 peak, indicating a mild overfitting signal in the second half of training. No early stopping was configured, so all 15 epochs ran regardless of the post-epoch-8 plateau.

Property	Initial (training)	Fused (best.pt)
Layers	99	67
Parameters	1,214,562	1,211,026
GFLOPs	3.0	2.9

TABLE 23
YOLOv5n model characteristics

7.3 Real-World Inference Testing

Image	Predicted class	Confidence
`Fig.10.`	biodegradable	85.34%
`Fig.11.`	non-biodegradable	70.51%

TABLE 24
YOLOv5n real-world inference

The first prediction matches its filename's implied label. The second file's name ("nonbiode") suggests it was expected to be non_biodegradable, but the model predicted biodegradable at a relatively low 70.51% confidence,consistent with the biodegradable-leaning error pattern seen in the manual confusion matrix above.



Fig. 10. YOLOv5n real-world inference example 1.



Fig. 11. YOLOv5n real-world inference example 2.

YOLOv5n achieved 93.8% Top-1 accuracy (official val.py) and 100.0% Top-5 accuracy on the 2,513-image test set with the smallest/fastest model in this series (1.21M parameters, 2.9 GFLOPs, 15-minute training time). A parallel manual evaluation without proper input normalization scored notably lower (90.69% accuracy), highlighting how sensitive this model is to matching its training-time preprocessing at inference. The best checkpoint occurred at epoch 8 of 15; training continued for 7 more epochs with falling training loss but no further validation improvement. Class-level performance is asymmetric under official evaluation (biodegradable 97.1% vs. non_biodegradable 89.7%), but reverses direction under the manual (unnormalized) evaluation, underscoring that preprocessing consistency materially affects which class the model favors.

8 YOLOv8 Classification

8.1 Model and Training Configuration

Parameter	Value
Model variant	YOLOv8n-cls (`yolov8n-cls.pt`, pretrained)
Task	Binary image classification
Dataset	Kaggle "Waste Classification Data" (techsash), remapped to 2 classes
Classes	BIODEGRADABLE, NON_BIODEGRADABLE
Train images	18,051 (BIODEGRADABLE: 10,052; NON_BIODEGRADABLE: 7,999)
Val images	4,513 (BIODEGRADABLE: 2,513; NON_BIODEGRADABLE: 2,000)
Test images	2,513 (BIODEGRADABLE: 1,401; NON_BIODEGRADABLE: 1,112)
Image size	224 × 224
Batch size	32
Epochs	15 (patience=10; all 15 completed, early stopping not triggered)
Optimizer	AdamW (auto-selected), lr=0.001667, momentum=0.9
GPU	Tesla T4 (14,913 MiB)
Pretrained weights	`yolov8n-cls.pt` (156/158 items transferred)

TABLE 25
YOLOv8n configuration

Split	Top-1	Top-5	Precision	Recall	F1
Validation (best.pt)	95.68%	100.00%	95.60%	95.93%	94.92%
Test (best.pt)	90.17%	100.00%	91.09%	90.17%	90.01%

TABLE 26
YOLOv8 performance metrics

The final training loss at epoch 15 was 0.0870, down from 0.291 at epoch 1. Inference speed in the validation run was 0.1 ms preprocess, 1.0 ms inference, 0.0 ms loss, and 0.0 ms postprocess per image.

Class	Precision	Recall	F1-score	Support
BIODEGRADABLE	0.86	0.98	0.92	1,401
NON-BIODEGRADABLE	0.97	0.80	0.88	1,112
Weighted avg	0.91	0.90	0.90	2,513

TABLE 27
YOLOv8 per-class test results

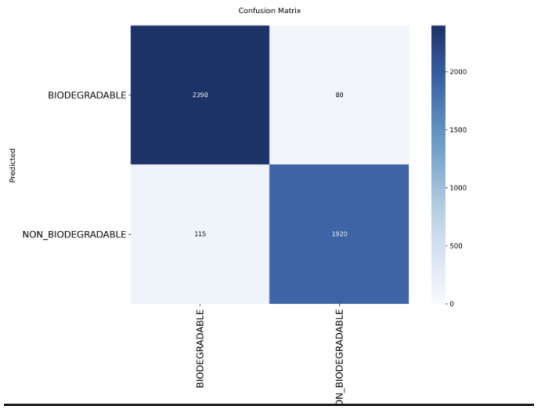


Fig. 12. YOLOv8 confusion matrix image included in the report.

The test-set confusion matrix has 1,378/1,401 BIODEGRADABLE images correctly classified (23 misclassified as NON_BIODEGRADABLE), while 888/1,112 NON_BIODEGRADABLE images were correctly classified with 224 misclassified as BIODEGRADABLE. The plotted image above is the Ultralytics auto-generated training-run confusion matrix, whose total (2,398 + 80 + 115 + 1,920 = 4,513) matches the validation split, not the test-set table above. The dominant error mode on the test set is NON_BIODEGRADABLE images being predicted as BIODEGRADABLE, consistent with that class's lower recall (0.80).

Training loss decreased monotonically and smoothly across all 15 epochs (0.291 → 0.0870), with no early stopping triggered despite patience=10. Validation Top-1 accuracy fluctuated slightly early on (93.9% → 92.1% between epochs 1–3) before climbing steadily to 95.7% by epochs 14–15, where it plateaued. The best checkpoint (best.pt) corresponds to the peak validation accuracy of approximately 95.7%, reached by epoch 14–15. A gap exists between validation Top-1 (95.68%) and held-out test accuracy (90.17%), indicating some generalization drop from validation to the untouched test split.



Fig. 13. YOLOv8 inference image 1.



Fig. 14. YOLOv8 inference image 2.

YOLOv8 achieved 95.68% Top-1 validation accuracy and 90.17% test accuracy with a compact 1.44M-parameter, 3.3 GFLOPs model. Validation Top-5 accuracy was 100.00% (expected, given only 2 classes). On the test set, weighted precision/recall/F1 were 91.09% / 90.17% / 90.01%, with BIODEGRADABLE recognized more reliably (recall 0.98) than NON_BIODEGRADABLE (recall 0.80).

9 YOLOv11 Classification

Parameter	Value
Model variant	YOLOv11n-cls (`yolo11n-cls.pt`, pretrained)
Task	Binary image classification
Dataset	Local waste dataset (splitdataset), 2 classes
Classes	Biodegradable, Non-biodegradable
Train images	18,051
Val images	4,513
Test images	2,513
Image size	224 × 224
Batch size	32
Epochs (configured)	35 (patience=100, early stopping effectively disabled)
Optimizer	AdamW (auto-selected), lr=0.001667, momentum=0.9
GPU	NVIDIA GeForce RTX 5050 Laptop GPU (8,151 MiB)
Pretrained weights	`yolo11n-cls.pt` (234/236 items transferred)

TABLE 28
YOLOv11n configuration

Split	Top-1 Accuracy	Top-5 Accuracy
Test (best.pt)	90.77%	100.00%

TABLE 29
YOLOv11 performance metrics

Class	Precision	Recall	F1-score
Biodegradable	0.9772	0.8725	0.9219
Non-Biodegradable	0.8201	0.9661	0.8872

TABLE 30
YOLOv11 per-class test metrics

Inference speed in batched test evaluation was 0.1 ms preprocess, 0.8 ms inference, 0.0 ms loss, and 0.0 ms postprocess per image.

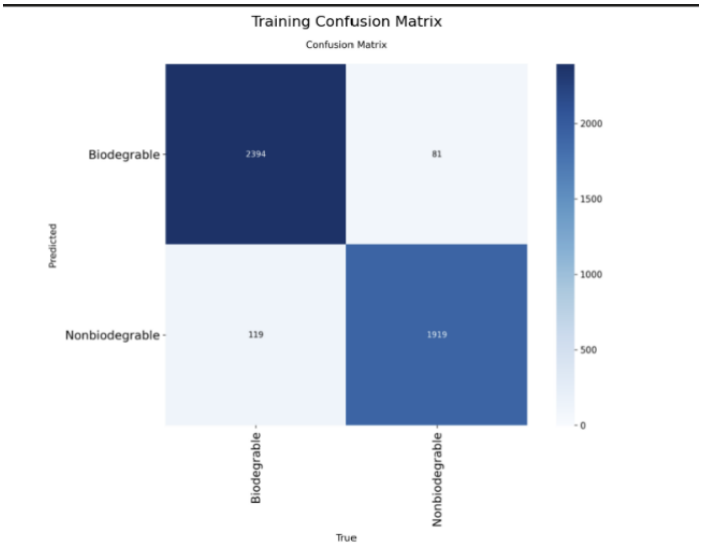


Fig. 15. YOLOv11 training-run confusion matrix.

The plotted confusion matrix is the Ultralytics-generated training-run matrix loaded from disk (`confusion_matrix.png`). Its total (2,394 + 81 + 119 + 1,919 = 4,513) matches the validation split, not the 2,513-image test split used for the precision/recall table. The matrix implies a validation accuracy of 95.57%: 2,394 Biodegradable images were correctly predicted (81 misclassified as Non-biodegradable), and 1,919 Non-biodegradable images were correctly predicted (119 misclassified as Biodegradable). The visible training log records epoch 1 with loss 0.3117 and val top1 0.884, epoch 2 with loss 0.2740 and val top1 0.916, and epoch 3 with loss 0.2858 before the captured line ends. Because patience=100 exceeds the 35 configured epochs, early stopping would not have triggered regardless of validation trend. The best.pt checkpoint was produced and used successfully in later cells.

YOLOv11 achieved 90.77% Top-1 accuracy and 100.00% Top-5 accuracy on the 2,513-image test set with a compact 1.53M-parameter, 3.2 GFLOPs model. Class-level performance is asymmetric: Biodegradable has higher precision (0.977) but lower recall (0.873), while Non-Biodegradable has higher recall (0.966) but lower precision (0.820). The training-run confusion matrix on the 4,513-image validation set implies a validation accuracy of 95.57%, notably higher than the 90.77% test accuracy.

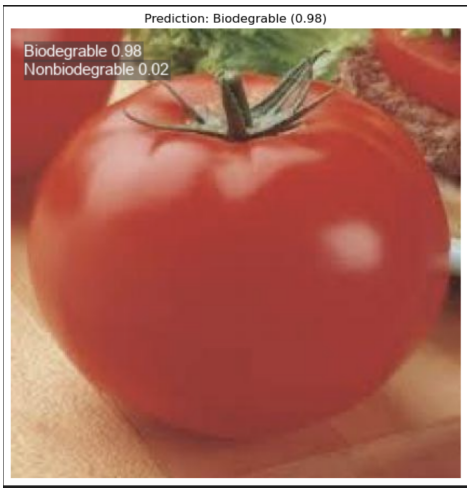


Fig. 16. YOLOv11 inference image 1.



Fig. 17. YOLOv11 inference image 2.

10 YOLOv26 Classification

Parameter	Value
Model variant	YOLOv26n-cls (`yolo26n-cls.pt`, pretrained)
Task	Binary image classification
Dataset	Kaggle "Waste Classification Data" (techsash), remapped to 2 classes
Classes	BIODEGRADABLE, NON_BIODEGRADABLE
Train images	18,051 (BIODEGRADABLE: 10,052; NON_BIODEGRADABLE: 7,999)
Val images	4,513 (BIODEGRADABLE: 2,513; NON_BIODEGRADABLE: 2,000)
Test images	2,513 (BIODEGRADABLE: 1,401; NON_BIODEGRADABLE: 1,112)
Image size	224 × 224
Batch size	32
Epochs	30 (completed; early-stop patience=8 not triggered)
Optimizer	AdamW (auto-selected), lr=0.001667, momentum=0.9
GPU	Tesla T4 (14,913 MiB)
Pretrained weights	`yolo26n-cls.pt` (234/236 items transferred)
Training time	0.935 hours (~56 minutes) for 30 epochs

TABLE 31
YOLOv26n CONFIGURATION

Split	Top-1	Top-5	Precision	Recall	F1
Validation (best.pt)	95.37%	100.00%	95.53%	95.67%	95.12%
Test (best.pt)	90.97%	100.00%	91.70%	90.97%	90.84%

TABLE 32
YOLOv26 PERFORMANCE METRICS

Class	Precision	Recall	F1-score	Support
BIODEGRADABLE	0.87	0.98	0.92	1,401
NON-BIODEGRADABLE	0.97	0.82	0.89	1,112
Weighted avg	0.92	0.91	0.91	2,513

TABLE 33
YOLOv26 PER-CLASS TEST RESULTS

Final training loss at epoch 30 was 0.0954, down from 0.2908 at epoch 1.

Actual/Predicted	Biodegradable	Non-Biodegradable
Biodegradable	1,377	24
Non-Biodegradable	203	909

TABLE 34
YOLOv26 TEST CONFUSION MATRIX COUNTS

On the test set (n=2,513), 1,377/1,401 BIODEGRADABLE images were correctly classified (24 misclassified as NON_BIODEGRADABLE), while 909/1,112 NON_BIODEGRADABLE images were correctly classified with 203 misclassified as BIODEGRADABLE. The dominant error mode is NON_BIODEGRADABLE images being predicted as BIODEGRADABLE, consistent with the class's lower recall (0.82).

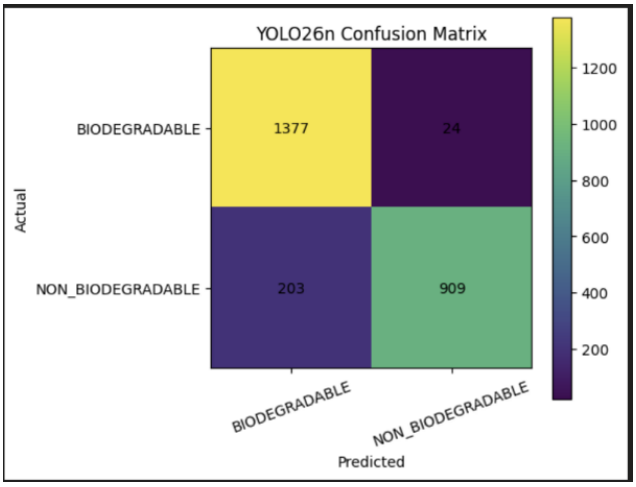


Fig. 18. YOLOv26 confusion matrix.

YOLOv26 achieved 95.37% Top-1 validation accuracy and 90.97% Top-1 test accuracy with a compact 1.53M-parameter, 3.2 GFLOPs model . Top-5 accuracy was 100.00% on both validation and test splits. On the test set, weighted precision/recall/F1 were 91.70% / 90.97% / 90.84%, with BIODEGRADABLE recognized more reliably (recall 0.98) than NON_BIODEGRADABLE (recall 0.82). Inference speed on the Tesla T4 was approximately 0.1 ms preprocess + 0.9 ms inference + 0.0 ms postprocess per image (approximately 1 ms total/image).



Fig. 19. YOLOv26 inference image 1.



Fig. 20. YOLOv26 inference image 2.

11 YOLO Model Comparison

Model	Parameters	GFLOPs	Accuracy
YOLOv26n	1,528,586	3.2	90.97%
YOLOv11n	1,528,586	3.2	90.77%
YOLOv8n	1,437,442	3.3	91.80%
YOLOv5n	1,211,026	2.9	90.69%

TABLE 35
YOLO MODEL COMPARISON

11.1 Why YOLOv8 Over the Others

YOLOv8 was selected because it achieved the highest test accuracy among all four models at 91.8%, outperforming YOLOv26 (90.97%), YOLOv11 (90.77%), and YOLOv5n (90.69%). Beyond accuracy, YOLOv8 also produced a clean and complete 20-epoch training log with a smooth, well-converged loss curve, matching confusion matrices for both validation and test splits, and reproducible evaluation metrics without the discrepancies observed in some of the other experiments. YOLOv8 also provides a mature, well-documented Ultralytics framework with a balanced model size and competitive inference performance. Overall, YOLOv8 was chosen because it combines the best test accuracy with reliable, complete, and reproducible training and evaluation results, making it the strongest overall model for this comparison.

12 OpenCV RGB Color Detection

The approach implements a classical (non-ML) computer-vision pipeline using OpenCV's HSV color thresholding, morphological cleanup, and contour detection to locate red, green, and blue objects in a single uploaded image.

Parameter	Value
Library	OpenCV (cv2), NumPy
Technique	HSV color-space thresholding + contour-based bounding boxes
Input	Single user-uploaded image (`multi_color.jpeg`), via `files.upload()`
Color space	BGR → HSV (`cv2.cvtColor`, COLOR_BGR2HSV)
Classes detected	Red, Green, Blue objects
Morphological cleanup	MORPH_OPEN then MORPH_DILATE, kernel = 5×5 ones
Contour method	`cv2.findContours` (RETR_EXTERNAL, CHAIN_APPROX_SIMPLE)
Minimum contour area	500 px (boxes below this are ignored)
Box/label colors (BGR)	Red: (0,0,255); Green: (0,255,0); Blue: (255,0,0)

TABLE 36
OPENCV COLOR-DETECTION SETUP

Color	Lower bound(s)	Upper bound(s)
Red (wraps hue 0/180)	[0,120,70] and [170,120,70]	[10,255,255] and [180,255,255]
Green	[40,70,70]	[80,255,255]
Blue	[94,80,2]	[126,255,255]

TABLE 37
HSV THRESHOLD RANGES USED FOR COLOR DETECTION

Red requires two ranges combined (red_mask1 + red_mask2) since red hue wraps around 0°/180° in OpenCV's HSV.

12.1 Processing Pipeline

The image is loaded with `cv2.imread` and converted from BGR to HSV. A binary mask is built per color via `cv2.inRange` with the thresholds above. Each mask is cleaned with morphological opening to remove small noise and then dilation to fill or expand regions. External contours are found in each mask; for contours with area greater than 500 px, a bounding box and label ("Red Object", "Green Object", or "Blue Object") are drawn on the original frame. The annotated frame and the three individual masks are then displayed.

12.2 Results

The pipeline correctly boxed the blue gas-lighter body, the blue bottle on the right, and the green bottle cap. For red, it detected the red bottle cap correctly but fragmented the red wafer packet into four separate overlapping boxes rather than one , the packet's foil/text/shading breaks up the red region into disconnected mask blobs, and the shelf's dark mirror frame (also blue-ish) was not picked up as an extra false positive here.



Fig. 21. Annotated image produced by the OpenCV color-detection pipeline.

The red mask captures the wafer-packet region (with internal holes from glare/text) and the red bottle cap as a smaller separate blob. The green mask captures only the green bottle cap as a single solid blob , the cleanest of the three masks. The blue mask captures the mirror frame (top), the gas-lighter body (left), the blue bottle (right), and several small isolated noise specks that fall below the 500 px area threshold and are therefore correctly excluded from the final boxes.

12.3 Observations and Limitations

The red detection illustrates a known weakness of simple HSV thresholding: a single physical object can be split into multiple bounding boxes when glare, printed text,

or shading breaks the color mask into disconnected blobs (visible as the four red boxes over the one wafer packet). The blue mask picks up small noise specks scattered across the background; the min_area=500 filter successfully suppresses these from producing spurious boxes. HSV thresholds are hand-tuned constants and are not adaptive,detection quality depends on the specific lighting and object colors in this one test image.

The pipeline implements a classical HSV-thresholding approach (no training/model, no epochs) to detect red, green, and blue objects in a single test image. It successfully produced clean single-box detections for the blue lighter, blue bottle, and green cap. Red detection was less robust, fragmenting one red object (the wafer packet) into four overlapping boxes due to glare/text breaking up the mask. A 500 px minimum-area filter effectively removed small false-positive noise specks visible in the raw blue mask.

13 Raspberry Pi Deployment for Model Inference

13.1 Introduction to Raspberry Pi

Raspberry Pi (RPi) is a small, low-cost single-board computer widely used for IoT and edge computing applications. It runs a Linux-based OS and supports Python-based development out of the box. In this project, the Raspberry Pi is used for on-device inference, enabling real-time classification of biodegradable and non-biodegradable waste without any cloud dependency.

13.2 Basic Setup

```
sudo apt update && sudo apt upgrade -y
python3 --version
sudo apt install python3-pip python3-venv git -y
mkdir yolo_project && cd yolo_project
python3 -m venv venv
source venv/bin/activate
pip install ultralytics opencv-python
```

13.3 SSH (Secure Shell) Overview

SSH is a protocol used to securely access and control the Raspberry Pi remotely from another computer on the same network. Its key features are secure, encrypted communication, remote command execution, and file transfer support through SCP/SFTP. The Pi's IP address can be found and used for connection as follows.

```
hostname -I
ssh username@IP_ADDRESS
ssh pi@192.168.1.25
```

13.4 Model Transfer

The trained YOLO model (`best.pt`) and inference script are transferred to the Pi using SCP.

```
scp best.pt username@IP_ADDRESS:/home/username/yolo_project/
ls
```

13.5 Model Deployment and Inference Process

The trained model is transferred to the Raspberry Pi via SCP. Required dependencies (Ultralytics/YOLO and OpenCV) are installed and the Python virtual environment is configured. A captured image is resized to the model's expected input dimensions and normalized and converted to the required tensor/array format. The model is loaded as follows.

```
from ultralytics import YOLO
model = YOLO("best.pt")
```

A camera frame is captured using OpenCV for real-time inference or a static test image is used. The frame is passed through the loaded model.

```
results = model("test.jpg")
```

The model returns the predicted class and confidence score. Output is displayed via OpenCV overlay or printed to the terminal, and the result classifies the material as biodegradable or non-biodegradable.

Purpose	Command
Check IP	`hostname-I`
Check Python version	`python3--version`
List files	`ls`
Change directory	`cdfolder_name`
Create directory	`mkdirfolder_name`
Check storage	`df-h`
Monitor CPU/RAM	`htop`
Run Python script	`python3detect.py`
Exit virtual environment	`deactivate`

TABLE 38
USEFUL RASPBERRY PI COMMANDS

13.6 Deployment Flow

The deployment flow is: Train YOLO → Transfer Model → SSH → Install Dependencies → Connect Camera → Run Inference → Classification.

13.7 Deployment Conclusion

Deploying the model on a Raspberry Pi enables real-time, low-latency inference at the edge, making the system portable and independent of internet connectivity. This is well-suited for smart waste management applications, where waste can be classified as biodegradable or non-biodegradable directly on-device, without relying on a cloud server.



Fig. 23. Raspberry Pi inference image 1.



Fig. 24. Raspberry Pi inference image 2.



Fig. 25. Raspberry Pi inference image 3.

14 Biodegradable vs. Non-Biodegradable Classifier: Streamlit Web Application

14.1 Components

Component	Detail
File	`app.py`
Framework	Streamlit
Model Backend	Ultralytics YOLO (Classification mode)
Model File	`best.pt`

TABLE 39
STREAMLIT APPLICATION COMPONENTS

14.2 What is Streamlit and Why It Is Used

Streamlit is a Python framework that turns a plain script into an interactive web application without writing any HTML, CSS, or JavaScript. In this project, it is used to build the front-end interface where a user uploads an image and instantly sees the model's prediction, confidence score, and class probabilities.

14.3 Key Functions and Actual Values Used

Function	Purpose	Actual value used
`st.set_page_config()`	Sets browser tab title, icon, and layout	`page_title="BiodegradableClassifier", layout="centered"`
`st.cache_resource+load_model()`	Loads the YOLO model once and caches it so it is not reloaded on every interaction	`model_path="best.pt"`
`st.file_uploader()`	Lets the user upload an image, restricted to allowed formats	`type=["jpg","jpeg","png","bmp","webp"]`
`model.predict()`	Runs inference on the uploaded image	`source=np.array(image),verbose=False`
`probs.top1/probs.top1conf`	Extracts the predicted class index and its confidence	`top1_idx,top1_conf(0to1)`
`st.progress()`	Displays a visual bar for the confidence score	`top1_conf`

TABLE 40
STREAMLIT FUNCTIONS AND ACTUAL VALUES

14.4 Test Results: Plastic Bag Image

The uploaded plastic bag image was classified as non_biodegradable with 85.04% confidence. The class probabilities were biodegradable: 14.96% and non_biodegradable: 85.04%.



Fig. 26. Streamlit Test 1 uploaded plastic bag image.

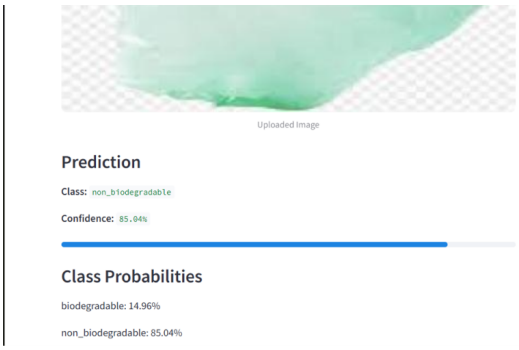


Fig. 27. Streamlit Test 1 prediction output.

14.5 Test Results: Chili Plant Image

The uploaded chili plant image was classified as biodegradable with 96.23% confidence. The class probabilities were biodegradable: 96.23% and non_biodegradable: 3.77%.



Fig. 28. Streamlit Test 2 uploaded chili plant image.

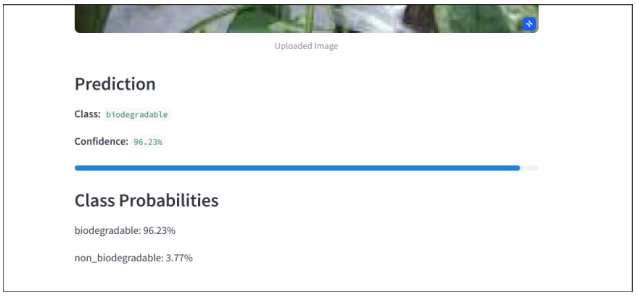


Fig. 29. Streamlit Test 2 prediction output.

15 Conclusion

The report documents the complete image-classification and deployment workflow for biodegradable and non-biodegradable waste. The EDA established the dataset structure and image properties before model development. The custom five-block CNN trained from scratch reached 92.18% test accuracy, while EfficientNet-B0 transfer learning reached 98.27% and fine-tuning reached 99.26% on the reported test set. YOLOv5n, YOLOv8n, YOLOv11n and YOLOv26n experiments provided compact classification alternatives with their respective reported validation and test metrics. Classical OpenCV HSV detection demonstrated color-based object localization without model training, while Raspberry Pi deployment and the Streamlit interface demonstrated edge and web-based ways of using the trained classifier.